



Lecture 2: Performance, Power- Wall-ILP & Hazards



Defining Performance

- ❑ "Faster" depends on perspective: a desktop user measures response time, while a datacentre operator measures jobs completed per hour.
- ❑ Architects define performance precisely so reasonable users don't disagree on what "faster" means.

RESPONSE TIME (LATENCY)

$$\text{Performance}(X) = 1 / \text{Execution_time}(X)$$

- How long a single task takes from start to finish.
- Matters for interactive use page loads, key-presses, sensor responses.

THROUGHPUT (BANDWIDTH)

Tasks completed per unit time

- How many tasks finish per second.
- Matters for servers, batch pipelines, and any workload bottlenecked by aggregate work, not single-task delay.

$$\text{SPEEDUP} \rightarrow \text{Speedup}(X \text{ over } Y) = \text{Execution_time}(Y) / \text{Execution_time}(X)$$

The CPU Performance Equation

➤ CPU Time = $IC \times CPI \times T_c$

➤ IC - Instruction Count

- How many instructions the program executes.
- Controlled by the compiler and the ISA.

➤ CPI - Cycles per Instruction

- Average clock cycles each instruction needs.
- Controlled by the microarchitecture (pipelining, caches, hazards).

➤ T_c - Clock Cycle Time

- Seconds per cycle.
- Set by circuit design, process technology, and the longest pipeline stage.

➤ To make a chip faster, we must reduce at least one of these factors without inflating the others.

Benchmarks & Misleading Metrics

❑ How do we compare two real processors fairly?

- With benchmarks (standard workloads) that everyone runs the same way. But some popular metrics are traps.

❑ SPEC CPU suites - diverse, audited integer and floating-point workloads.

❑ Application-specific benchmarks (TPC for transactions, MLPerf for ML).

❑ Misleading Metrics:

- MIPS - ignores what each instruction actually accomplishes.
- Peak MFLOPS - rate the chip can theoretically achieve, rarely sustained.
- Clock frequency alone - says nothing about CPI or IC.
- Arithmetic mean of speedups - biased by the largest ratios.

❑ Trustworthy

- Wall-clock time on the real workload of interest.

Amdahl's Law

- ❑ If you only speed up part of a program, the overall improvement is capped by the part you did NOT touch. Amdahl's Law makes this precise:

$$\text{Speedup_overall} = 1 / ((1 - F) + F / S)$$

VARIABLES

- F = fraction that is improved.
- S = speedup of that improved portion.
- $(1 - F)$ = the un-touched fraction - the bottleneck.

EXAMPLE

- If 80% of a program is parallelised perfectly ($F = 0.8, S \rightarrow \infty$):
 - ❖ The maximum overall speedup is $1 / (1 - 0.8) = 5\times$. The serial 20% caps the gain.

- ❑ Implication - optimise the common case first.

Power wall

- ❑ For ~30 years, chips got faster mainly by raising clock frequency (MHz → GHz)
- ❑ Around 2005 this hit a hard limit: the power wall
- ❑ Faster switching burns more power, and that power turns into heat in a tiny area
- ❑ There's a cooling ceiling ($\sim 100\text{--}150\text{ W/cm}^2$) you physically can't exceed
- ❑ Fix: stop chasing one fast core; build many moderate cores instead

Power wall Equations

$$P_{\text{dynamic}} \approx \alpha \cdot C \cdot V^2 \cdot f$$

- ❑ α = activity factor (fraction of transistors switching per cycle)
- ❑ C = switched capacitance (gates and wires being charged/discharged)
- ❑ V = supply voltage. It is squared, so it dominates the cost
- ❑ f = clock frequency
- ❑ Raising f forces V up too $\rightarrow V^2$ makes it expensive

Why frequency raises voltage

- ❑ Higher clock needs a shorter gate delay (transistors must switch faster)

- ❑ Gate delay $\propto \frac{C.V}{(V-V_{th})^\alpha}$

- ❑ V_{th} = threshold voltage. It is a fixed floor that doesn't scale down

- ❑ To switch faster, you need a bigger gap above V_{th} → push V up

- ❑ So: higher $f \Rightarrow$ higher $V \Rightarrow$ power rises with V^2 and f together

Example

❑ A single processor core is characterized by an activity factor $\alpha = 0.25$, a switched capacitance $C = 100$ nF, a supply voltage $V = 1.0$ V, and a clock frequency $f = 4$ GHz. Using the dynamic power relation $P = \alpha \cdot C \cdot V^2 \cdot f$, answer the following.

❑ Questions:

- ❖ Determine the baseline dynamic power dissipated by the core.
- ❖ To gain more performance, the clock is raised to 5 GHz. Because the transistors must switch faster, the core now requires a higher supply voltage of $V = 1.15$ V. Determine the new power dissipation and comment on the result.
- ❖ Alternatively, two identical cores are used, each operating at $V = 0.8$ V and $f = 3.2$ GHz. Determine the total power dissipation and the combined performance relative to the baseline.

Solution

□ Baseline Power

- ❖ $P = 0.25 \times (100 \times 10^{-9}) \times (1.0)^2 \times (4 \times 10^9)$

- ❖ $P = 100 \text{ W}$ ← this is our power budget

□ The frequency trap

- ❖ Frequency alone would scale linearly: $100 \times (5/4) = 125 \text{ W}$

- ❖ But voltage rises too:

- ❖ $P = 100 \times (1.15)^2 \times (5/4) = 100 \times 1.3225 \times 1.25 = 165.3125 \text{ W}$

- ❖ 25% more speed → 65% more power dissipated. This is the wall.

□ The multicore escape

- ❖ Per core: $P = 100 \times (0.8)^2 \times (0.8) = 51.2 \text{ W}$

- ❖ Two cores: $\approx \mathbf{102 \text{ W}}$ ($\approx$ baseline budget)

- ❖ Performance = $2 \times 3.2 \text{ GHz}$ vs $1 \times 4 \text{ GHz} = \mathbf{1.6 \times \text{the work}}$

- ❖ Cutting V pays off big because it's squared

Design	Power	Performance
Baseline (1 core)	100 W	1.0×
Overclock to 5 GHz	165 W	1.25×
Two cores @ 3.2 GHz	102 W	1.6×

Introduction to ILP

- ❑ ILP : overlapping the execution of independent instructions
- ❑ A program is a sequence of instructions in source-order but the underlying hardware is free to execute them in any order that respects data and control dependencies.
- ❑ ILP is the architect's discipline of finding and exploiting independent operations so multiple complete each cycle.

Sequential:

I1	I2	I3	I4	I5	I6	I7	I8
----	----	----	----	----	----	----	----

With ILP:

I1	I3	I5	I7
I2	I4	I6	I8

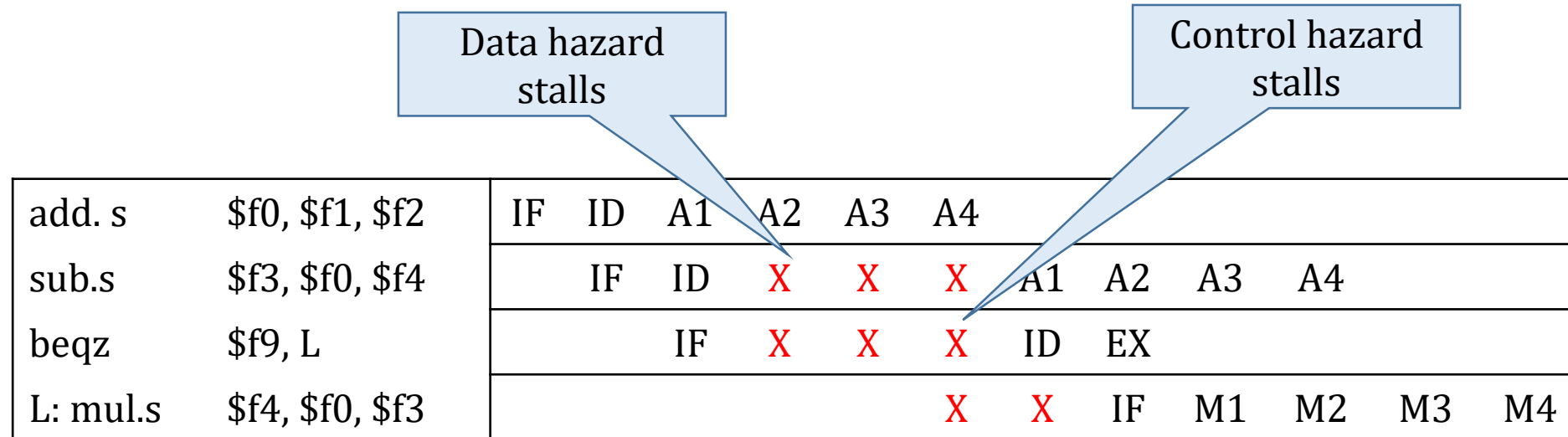
Half the time same work.

How Processors Exploit ILP

Exploiting ILP requires both compiler discipline and hardware machinery. Real processors combine several techniques:

1. **Pipelining**
 - Overlap the stages of consecutive instructions fetch I3 while I2 is in decode and I1 is in execute.
2. **Superscalar issue**
 - Issue multiple instructions in the same cycle through duplicated functional units.
3. **Out-of-order execution**
 - Hardware reorders instructions dynamically to keep functional units busy while respecting dependencies.
4. **Speculative execution**
 - Predict branches and execute beyond them, rolling back if the prediction was wrong.
5. **Register renaming**
 - Eliminate false dependencies (WAR, WAW) by mapping architectural registers onto a larger physical pool.
6. **Compiler scheduling**
 - Static reordering by the compiler to expose independent operations to the hardware.

Motivation- Stalls due to Pipeline



- Eliminating data hazards
 - When only the executable is available – cannot reschedule the instruction in the code
 - Dynamic Scheduling
 - Out of Order (OoO) execution
- Eliminating control hazards
 - Branch prediction

Multiple Issue

- ❑ Single-issue pipeline processor issues 1 instruction per cycle

IF	ID	EX	M	WB		
	IF	ID	EX	M	WB	
		IF	ID	EX	M	WB

- What's next: Multiple-issue pipeline processor: more than 1 issue of instruction per cycle

IF	ID	EX	M	WB		
IF	ID	EX	M	WB		
	IF	ID	EX	M	WB	
	IF	ID	EX	M	WB	
		IF	ID	EX	M	WB
		IF	ID	EX	M	WB

Multiple-Issues Techniques

❑ Dynamic Techniques

- ❖ Hardware based ILP
- ❖ Mostly desktops and servers
- ❖ Intel Core Series, MIPS32, AMD Ryzen, etc.



❑ Static Techniques

- ❖ Software based (compiler)
- ❖ Mostly in embedded or DSP devices
- ❖ TriMedia TM32, Texas Instruments DSPs



What is instruction Level Parallelism (ILP) ?

❑ ILP is a technique where instructions can execute partially or fully in parallel

❖ Pipelining (partially)

❖ Multiple issue (Fully)

IF	ID	EX	M	WB		
	IF	ID	EX	M	WB	
		IF	ID	EX	M	WB

IF	ID	EX	M	WB		
IF	ID	EX	M	WB		
	IF	ID	EX	M	WB	
	IF	ID	EX	M	WB	
		IF	ID	EX	M	WB
		IF	ID	EX	M	WB

Thank you